

Terraform Notes

Terraform Overview

Terraform is an **Infrastructure as Code (IaC)** tool created by **HashiCorp** that allows you to **provision, manage, and version** cloud infrastructure efficiently using **declarative configuration files**.

What is Terraform?

- A tool to **write, plan, and apply** infrastructure.
- Uses **HCL (HashiCorp Configuration Language)** for defining resources.
- Works with **many cloud providers**: AWS, Azure, GCP, etc.
- Platform-agnostic and open-source.

Terraform Workflow

1. Write Configuration

- a. Files ending in `.tf` (e.g., `main.tf`, `variables.tf`)
- b. Define provider, resource, variable, output, etc.

2. Initialize

```
terraform init
```

- a. Downloads required provider plugins.

3. Plan

```
terraform plan
```

- a. Shows the changes Terraform will make without applying them.

4. Apply

<https://www.linkedin.com/in/rishika-singh-sikarwar/>

terraform apply

- a. Provisions or updates infrastructure as defined in .tf files.

5. Destroy

terraform destroy

- a. Deletes all resources created by Terraform.

Key Terraform Components

Component

provider

resource

variable

output

module

Example: Simple AWS EC2

```
provider "aws" {  
  region = "us-east-1"
```

<https://www.linkedin.com/in/rishika-singh-sikarwar/>

```
}
```

```
resource "aws_instance" "example" {  
  ami          = "ami-12345678"  
  instance_type = "t2.micro"  
}
```

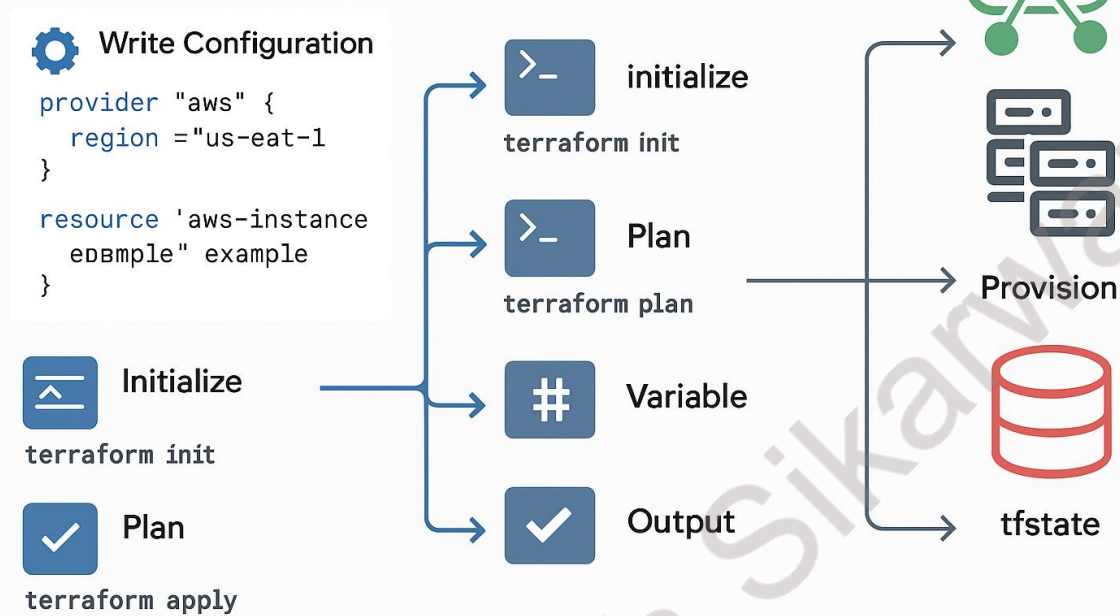
State Management

- Terraform tracks infrastructure in a `.tfstate` file.
- This file stores the current state of resources.
- Required for consistent provisioning and updates.

Benefits of Using Terraform

- **Infrastructure as Code** (version-controlled)
- **Reusable** (via modules and variables)
- **Consistent & Predictable**
- **Multi-cloud support**
- Supports **remote backends**, **locking**, and **state sharing**

Terraform Overview



Modifying Resources in Terraform

Modifying resources means making changes to existing infrastructure by updating the .tf files and reapplying them using Terraform.

✓ 1. How to Modify Resources

- Simply **edit the configuration** in your .tf files.
- Examples of changes:
 - Updating EC2 instance type
 - Changing tags
 - Adding/removing properties (e.g., EBS volume)

Example

Original Configuration

```
resource "aws_instance" "web" {  
  ami           = "ami-0abcdef1234567890"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "MyInstance"  
  }  
}
```

Modified Configuration

```
resource "aws_instance" "web" {  
  ami           = "ami-0abcdef1234567890"  
  instance_type = "t3.micro"           # changed instance type  
  tags = {  
    Name = "UpdatedInstance"         # changed tag  
  }  
}
```

2. Preview Changes

Before applying:

```
terraform plan
```

This shows a diff-like output of what will change:

- ~ means **update in-place**
- + means **addition**

- - means **deletion**

3. Apply the Changes

To make changes take effect:

```
terraform apply
```

Terraform will prompt for approval unless auto-approved.

4. Types of Modifications

Type	Result
Minor changes (tags, type)	In-place update (~)
Critical changes (AMI ID)	Resource recreation (- +/)
Remove property	Update or destroy+create

5. Important Notes

- Some changes **require resource replacement** (destroy and re-create).
- If a resource must be recreated, Terraform shows:

```
-/+ resource "aws_instance" "web" {
```

6. Best Practices

- Always run `terraform plan` before `apply`.

- Use version control (e.g., Git) to track changes.
- Keep backups of .tfstate and use remote state if working in teams.

Deleting Resources in Terraform

Terraform allows you to remove infrastructure safely by updating your configuration files and syncing with the actual cloud environment.

☒ 1. Two Main Ways to Delete Resources

◆ A. Delete from .tf File (Recommended)

1. Open your .tf file.
2. **Remove** the resource block you want to delete.

```
# Delete this block completely
resource "aws_s3_bucket" "demo" {
  bucket = "my-demo-bucket"
  acl     = "private"
}
```

3. Run:

```
terraform plan
terraform apply
```

Terraform will detect that the resource was removed from the configuration and show:

- aws_s3_bucket.demo will be destroyed

◆ B. Use terraform destroy (Full Cleanup)

This deletes **all** resources in the current project.

terraform destroy

- It reads the .tfstate file and deletes everything it manages.
- Use carefully, especially in production environments.

To confirm automatically:

terraform destroy -auto-approve

🔗 Optional: Delete a Specific Resource Only

Use the -target flag with terraform destroy:

terraform destroy -target=aws_s3_bucket.demo

- Deletes only the targeted resource, nothing else.

🔍 What Happens Internally?

- Terraform checks .tfstate against .tf code.
- If a resource is missing from .tf, it schedules it for **deletion**.
- Cloud provider (AWS, etc.) is then called to **terminate** the resource.

⚠️ Precautions Before Deleting

- Always run:

`terraform plan -destroy`

to preview what will be removed.

- If you delete resources manually in the console (not through Terraform), you may break state.
- Use `terraform state rm` to remove manually deleted resources from state.

Useful Commands

Command	Description
<code>terraform apply</code>	Deletes if resource is removed from .tf
<code>terraform destroy</code>	Deletes everything
<code>terraform destroy -target=resource</code>	Deletes only one
<code>terraform state rm resource</code>	Removes resource from state file only

Referencing Resources in Terraform

In Terraform, **referencing** allows one resource to use information (like ID, name, or IP) from another resource. This creates **dependency chaining** and ensures correct provisioning order.

Why Reference?

- Use output from one resource in another.
- Prevent hardcoding.
- Ensure dependency-based execution.

☑ Syntax for Referencing

<resource_type>.<resource_name>.<attribute>

Example:

aws_instance.web.public_ip

This refers to the **public IP** of a resource of type `aws_instance` named `web`.

🔗 Basic Example: Referencing Security Group in EC2

```
resource "aws_security_group" "web_sg" {
  name = "web-sg"
  ...
}

resource "aws_instance" "web" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"
  vpc_security_group_ids = [aws_security_group.web_sg.id] #
referencing SG
}
```

Terraform automatically knows to create the security group **before** the EC2 instance.

📄 Referencing Outputs

You can define outputs to make values usable in other modules or externally:

outputs.tf

```
output "instance_ip" {
  value = aws_instance.web.public_ip
}
```

<https://www.linkedin.com/in/rishika-singh-sikarwar/>

```
}
```

Then retrieve using:

```
terraform output instance_ip
```

Chained Referencing (Nested)

Resources can reference others which themselves reference others:

```
resource "aws_eip" "ip" {  
  instance = aws_instance.web.id  
}
```

Here, `aws_eip` is directly tied to the EC2 instance by using its `id`.

Important Attributes Commonly Referenced

Attribute	Description
<code>.id</code>	Unique resource identifier
<code>.arn</code>	Amazon Resource Name
<code>.public_ip</code>	Public IP for EC2 or EIP
<code>.tags</code>	Map of tags applied
<code>.name</code>	(If name is defined/tagged)

Best Practices

- Always use references instead of hardcoded IDs or values.
- Let Terraform handle dependencies through referencing.
- Avoid circular dependencies ($A \rightarrow B \rightarrow A$).

<https://www.linkedin.com/in/rishika-singh-sikarwar/>

Terraform Files Overview

Terraform uses specific types of files to organize infrastructure as code. These files help in managing resources, variables, outputs, and more.

◇ 1.main.tf

Main configuration file

- Defines all the **resources** and **providers**.
- Can contain other blocks like variable, output, etc., but usually focused on infrastructure.

Example:

```
provider "aws" {  
  region = "ap-south-1"  
}  
  
resource "aws_instance" "web" {  
  ami           = "ami-0abcdef1234567890"  
  instance_type = "t2.micro"  
}
```

◇ 2.variables.tf

Defines **input variables** to make the configuration **reusable and dynamic**.

Example:

```
variable "instance_type" {  
  type    = string  
  default = "t2.micro"  
}
```

You can use the variable in `main.tf`:

```
instance_type = var.instance_type
```

◇ 3. `outputs.tf`

Defines the **output values** after resource creation—useful for returning public IPs, IDs, names, etc.

Example:

```
output "instance_ip" {  
  value = aws_instance.web.public_ip  
}
```

You can fetch it using:

```
terraform output instance_ip
```

◇ 4. `terraform.tfvars`

This file provides **actual values for input variables** defined in `variables.tf`.

Example:

```
instance_type = "t3.micro"
```

Terraform automatically reads this file when you run `terraform apply`.

◇ 5. `terraform.tfstate` (generated automatically)

- Stores the **current state** of your infrastructure.

- Used to track which resources Terraform manages.
- DO NOT manually edit this file.

◇ 6. terraform.lock.hcl (generated)

- Ensures consistent provider versions across team members.
- Helps in **locking** dependency versions to avoid mismatch.

Optional Files

File	Purpose
backend.tf	Configures remote state (e.g., S3, Terraform Cloud)
providers.tf	(Optional) Separates provider block from main.tf
.terraform/ /	Hidden folder storing plugins and metadata

Best Practices

- Keep code modular and readable.
- Use separate files for main.tf, variables.tf, and outputs.tf.
- Use terraform.tfvars to avoid hardcoding values.
- Always version-control .tf files, but exclude .tfstate if sensitive (or use .gitignore).

Terraform Practice Project: Launching an EC2 Instance on AWS

Objective:

Provision a **t2.micro EC2 instance** with a **security group** using Terraform.

<https://www.linkedin.com/in/rishika-singh-sikarwar/>

Folder Structure:

```
terraform-practice/  
├─ main.tf  
├─ variables.tf  
├─ outputs.tf  
└─ terraform.tfvars
```

Step-by-Step Code

main.tf

```
provider "aws" {  
    region = var.aws_region  
}  
  
resource "aws_instance" "my_ec2" {  
    ami            = var.ami_id  
    instance_type  = var.instance_type  
    key_name       = var.key_name  
  
    vpc_security_group_ids = [aws_security_group.my_sg.id]  
  
    tags = {  
        Name = "Terraform-EC2"  
    }  
}  
  
resource "aws_security_group" "my_sg" {  
    name           = "terraform-sg"  
    description    = "Allow SSH and HTTP"  
  
    ingress {  
        from_port = 22
```

<https://www.linkedin.com/in/rishika-singh-sikarwar/>

```

    to_port      = 22
    protocol     = "tcp"
    cidr_blocks  = ["0.0.0.0/0"]
  }

  ingress {
    from_port    = 80
    to_port      = 80
    protocol     = "tcp"
    cidr_blocks  = ["0.0.0.0/0"]
  }

  egress {
    from_port    = 0
    to_port      = 0
    protocol     = "-1"
    cidr_blocks  = ["0.0.0.0/0"]
  }
}

```

variables.tf

```

variable "aws_region" {
  default = "ap-south-1"
}

variable "instance_type" {
  default = "t2.micro"
}

variable "ami_id" {
  description = "AMI ID for the EC2 instance"
  default     = "ami-03f4878755434977f" # Amazon Linux 2 in ap-south-1
  (update if needed)
}

variable "key_name" {

```

```
description = "Name of the key pair to use for EC2"
}
```

terraform.tfvars

```
key_name = "your-existing-keypair-name"
```

Replace "your-existing-keypair-name" with your actual AWS EC2 key pair name.

outputs.tf

```
output "instance_id" {
  value = aws_instance.my_ec2.id
}

output "public_ip" {
  value = aws_instance.my_ec2.public_ip
}
```

Steps to Run the Project

1. Initialize the project

```
terraform init
```

2. Preview the plan

```
terraform plan
```

3. Apply the configuration

```
terraform apply
```

4. Verify Output

Terraform will show the EC2 instance_id and public_ip.

5. Destroy Resources (Cleanup)

```
terraform destroy
```

Terraform State & State Commands

What is the Terraform State?

- Terraform uses a file called `terraform.tfstate` to keep track of the real infrastructure it manages.
- It maps Terraform configuration to the actual resources in the cloud.
- Without this state file, Terraform **wouldn't know** what it has created or what needs to be changed.

Key Terraform State Commands

◇ `terraform state list`

Purpose: Lists all resources tracked in the current state file.

Usage:

```
terraform state list
```

Example Output:

```
aws_instance.my_ec2  
aws_s3_bucket.my_bucket
```

◇ terraform state show

Purpose: Shows detailed information about a specific resource in the state.

Usage:

```
terraform state show aws_instance.my_ec2
```

Output: Includes attributes like `ami`, `public_ip`, `tags`, etc.

◇ terraform state rm

Purpose: Removes a resource **from the state file only** (Terraform stops managing it).

Usage:

```
terraform state rm aws_s3_bucket.my_bucket
```

⚠ This **does not delete** the resource in AWS, it just removes Terraform's tracking of it.

◇ terraform state pull

Purpose: Downloads and prints the raw state file.

Usage:

```
terraform state pull
```

◇ terraform state push

Purpose: Uploads a local .tfstate file to a remote backend (used only with remote state).

Usage:

```
terraform state push my_state_file.tfstate
```

◇ terraform state mv

Purpose: Moves a resource from one name to another (e.g., rename or module restructure).

Usage:

```
terraform state mv aws_instance.old_name aws_instance.new_name
```

Location of State File

- **Local backend:** terraform.tfstate in project directory.
- **Remote backend:** Stored in S3, Terraform Cloud, etc.
 - Helps with team collaboration.
 - Supports **locking** to avoid conflicts.

Best Practices

- Never manually edit .tfstate.
- Use remote backends like S3 + DynamoDB (for state locking).

- Backup your state file before major changes.
- Use `terraform refresh` to update state with real-world changes.

Terraform Output

Terraform output allows you to **display useful information** after a successful `apply`—like an instance's IP, an S3 bucket name, or an ARN.

Why Use Outputs?

- Show results to users after provisioning.
- Pass values to other Terraform configurations (modules).
- Debug and verify infrastructure details.
- Use in automation/scripts.

Syntax

Declare Output in `outputs.tf`:

```
output "instance_ip" {  
  value = aws_instance.web.public_ip  
}
```

Access Output

After `terraform apply`, use:

```
terraform output
```

You'll see:

<https://www.linkedin.com/in/rishika-singh-sikarwar/>

```
instance_ip = "13.235.44.22"
```

To get a specific output:

```
terraform output instance_ip
```

Optional Output Arguments

Argument	Description
value	Required. The actual output expression
description	Helpful comment for understanding
sensitive	Hides the output (e.g., passwords)
depends_on	Forces output to wait for resource completion

Example:

```
output "db_password" {  
  value      = aws_secretsmanager_secret.db.secret_string  
  sensitive = true  
}
```

Output from a Module

In a **root module** (main project), you can get output from child modules:

```
module "network" {  
  source = "./vpc"  
}  
  
output "vpc_id" {  
  value = module.network.vpc_id  
}
```

}

Best Practices

- Use descriptive names: `vpc_id`, `instance_ip`, `bucket_name`.
- Mark secrets as `sensitive = true`.
- Keep outputs in a separate file: `outputs.tf`.

Target Resources in Terraform

The `-target` option allows you to apply, plan, or destroy **specific resources only** instead of the entire configuration.

1. When to Use Targeting

- For **testing** one resource at a time.
- To **quickly deploy** a single component.
- For **troubleshooting** or rerunning a failed resource.

Syntax

Apply a Single Resource

```
terraform apply -target=aws_instance.web
```

Plan a Specific Resource

```
terraform plan -target=aws_s3_bucket.my_bucket
```

◇ Destroy a Single Resource

```
terraform destroy -target=aws_security_group.allow_http
```

Resource Format

<resource_type>.<resource_name>

Example:

```
aws_instance.web
```

Important Notes

Rule	Explanation
Targeting is not recommended for routine use	It can bypass dependency resolution , leading to broken infrastructure.
Use for troubleshooting or isolated changes	Helpful in CI/CD pipelines and test environments.
Terraform still reads .tfstate	So changes will affect the tracked state.

Example

main.tf

```
resource "aws_s3_bucket" "demo" {  
  bucket = "terraform-demo-bucket"  
}  
  
resource "aws_instance" "web" {  
  ami           = "ami-12345678"  
  instance_type = "t2.micro"
```

```
}
```

To create **only the EC2 instance**:

```
terraform apply -target=aws_instance.web
```

Multiple Targets

You can target more than one resource:

```
terraform apply \  
-target=aws_instance.web \  
-target=aws_security_group.sg1
```

When Not to Use -target

- Don't rely on it for **regular deployments**.
- Avoid using in **modular or production** environments unless absolutely necessary.

Terraform Variables

Terraform **variables** let you **parameterize** your code so that it becomes reusable, flexible, and manageable.

1. Why Use Variables?

- Avoid hardcoding values.
- Make configs reusable across environments (dev, prod).
- Centralize configuration changes.

◇ 2. Types of Variables

Type	Description
string	Text values (e.g., instance type, region)
number	Numeric values
boolean	true or false
list	Ordered list (e.g., list of subnets)
map	Key-value pairs
object	Complex structured variables

🔗 3. Declaring Variables

In `variables.tf`:

```
variable "instance_type" {  
  description = "EC2 instance type"  
  type        = string  
  default     = "t2.micro"  
}
```

🎯 4. Using Variables in Your Code

In `main.tf`:

```
resource "aws_instance" "web" {  
  ami           = var.ami_id  
  instance_type = var.instance_type  
}
```

Use `var.<variable_name>` to reference it.

5. Providing Values

You can pass variable values in several ways:

terraform.tfvars

```
instance_type = "t3.micro"
ami_id        = "ami-12345678"
```

Inline CLI:

```
terraform apply -var="instance_type=t3.micro"
```

Environment Variable:

```
export TF_VAR_instance_type="t3.micro"
```

6. Variable Validation

You can add constraints:

```
variable "instance_type" {
  type = string
  validation {
    condition     = contains(["t2.micro", "t3.micro"],
var.instance_type)
    error_message = "Only t2.micro and t3.micro are allowed."
  }
}
```

7. Best Practices

- Use `variables.tf` for declarations.
- Use `terraform.tfvars` or environment-specific `.tfvars` files for values.
- Avoid hardcoding secrets—use environment variables or tools like Vault.

Recommended File Structure

```
main.tf
variables.tf
terraform.tfvars
outputs.tf
```

Basic Terraform Commands

Command	Description
<code>terraform init</code>	Initializes a working directory with Terraform configuration files. Downloads plugins/providers.
<code>terraform validate</code>	Validates the configuration syntax.
<code>terraform plan</code>	Shows what Terraform will do before applying.
<code>terraform apply</code>	Applies the configuration and provisions resources.
<code>terraform destroy</code>	Destroys the infrastructure created by Terraform.
<code>terraform fmt</code>	Formats configuration files to the standard style.
<code>terraform version</code>	Displays the Terraform version.
<code>terraform -help</code>	Shows help for Terraform CLI.

Configuration and Directory Commands

Command	Description
terraform workspace list	Lists all workspaces.
terraform workspace new <name>	Creates a new workspace.
terraform workspace select <name>	Switches to a specific workspace.
terraform workspace delete <name>	Deletes a workspace.

State Management Commands

Command	Description
terraform state list	Lists all resources in the current state.
terraform state show <resource>	Shows details of a resource in state.
terraform state rm <resource>	Removes a resource from the state file.
terraform state mv <source> <destination>	Moves a resource to a different name or module.
terraform state pull	Downloads the current state.
terraform state push	Uploads a local state file.

Debugging and Inspection

Command	Description
terraform graph	Generates a visual dependency graph.
terraform providers	Lists providers used in configuration.
terraform output	Shows outputs defined in the configuration.
terraform console	Opens an interactive console to evaluate expressions.

Cloud & Remote Backend

Command	Description
terraform login	Authenticates with Terraform Cloud.
terraform logout	Logs out of Terraform Cloud.
terraform cloud	Manages Terraform Cloud settings.

Plan and Apply with Options

Command	Description
terraform plan -out=tfplan	Saves the plan to a file.
terraform apply tfplan	Applies a saved plan.
terraform destroy -auto-approve	Destroys infrastructure without prompting.
terraform apply -auto-approve	Applies infrastructure changes without prompt.